Any of the token formats discussed in chapters 5 and 6 are suitable for generating API keys, with the username replaced by an identifier for the service or business that API usage should be associated with and the expiry time set to a few months or years in the future. Permissions or scopes can be used to restrict which API calls can be called by which clients, and the resources they can read or modify, just as you've done for users in previous chapters: the same techniques apply.

An increasingly common choice is to replace ad hoc API-key formats with standard JSON Web Tokens (JWTs). In this case, the JWT is generated by the developer portal with claims describing the client and expiry time, and then either signed or encrypted with one of the symmetric authenticated encryption schemes described in chapter 6. This is known as *JWT bearer authentication*, because the JWT is acting as a pure bearer token: any client in possession of the JWT can use it to access the APIs it is valid for without presenting any other credentials. The JWT is usually passed to the API in the Authorization header using the standard Bearer scheme described in chapter 5.

> **DEFINITION** In ***JWT bearer authentication*** a client gains access to an API by presenting a JWT that has been signed or encrypted by an issuer that the API trusts.

An advantage of JWTs over simple database tokens or encrypted strings, is that you can use public key signatures to allow a single developer portal to generate tokens that are accepted by many different APIs. Only the developer